Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Современные операционные системы и базы данных»

на тему:
«Подключение базы данных к сайту-портфолио»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверила
Преподаватель:
доцент, к.н., Галлини
Надежда Игоревна

Оценка:_____
Дата: 16.10.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Отчёт о выполнении

Цель работы - подключить SQLite к серверной части учебного сайта-портфолио, разобрать таблицы и параметризованные запросы, управлять материалами через админку и проверить, что записи остаются после перезапуска. Образец по заданию - серверная версия сайта «Сайт срочно». Готовый статический сайт двух предыдущих работ лежит в папке frontend-dev, её я и подключил к базе. Отдельный архив учебного комплекта к заданию не прилагался, поэтому сервер, схему и админку я собрал теми средствами, которые уже есть на компьютере: Node.js 22.15.0 и встроенный модуль node:sqlite. На этой версии модуль включается флагом --experimental-sqlite, его подставляет server/start.mjs.

Результат - главная страница получает опубликованные проекты из файла frontend-dev/data/portfolio.sqlite. Администратор входит по паролю, видит черновики, создаёт и меняет материалы. После остановки и нового запуска сервера те же записи на месте, а старый сеанс уже не действует. Ниже показано, как действие в браузере доходит до SQL. Пароль, cookie и файл базы в отчёт не входят.

1. Архитектура

Браузер к файлу базы не подключается. Он шлёт HTTP-запрос процессу Node.js. Сервер проверяет сеанс, поля и только потом выполняет SQL. Ответ снова уходит в браузер. Схема такая: браузер → HTTP API → server/app.mjs → server/store.mjs → SQLite → ответ.

Таблица 1 - компоненты приложения

Файл	**Назначение**
index.html, css/, js/, images/	Публичная страница прежнего портфолио
server/app.mjs	Маршруты, проверка входа и выдача страниц
server/store.mjs	Открытие базы, миграция и сохранение
server/schema.sql	Таблицы entries, admins, inquiries и индекс
server/admin.html, admin.js, admin.css	Вход, список материалов, редактор и заявки
server/site.js	Форма учебной заявки на главной

server/inspect.mjs	Чтение записей из консоли без изменения базы
server/read-projects.mjs	Выборка опубликованных проектов и два COUNT
server/sql-experiment.mjs	INSERT и UPDATE внутри транзакции с откатом
data/portfolio.sqlite	Файл базы, его не отдаёт публичный маршрут

Граница между браузером и SQL проходит на сервере. admin.js только собирает поля формы и вызывает fetch. Пароль в admin.js не записан: иначе он оказался бы в исходном тексте страницы. В таблицу admins попадает соль и scrypt-хеш. Проверка пароля, CSRF-токена и сеанса стоит в app.mjs, а prepare и run - в store.mjs. Публичный список берёт только строки с published = 1. Сеанс лежит в памяти процесса: выход, истечение срока или перезапуск его обрывают. Cookie стоит с флагом HttpOnly.

2. Таблицы и ограничения

Проекты, услуги и заметки лежат в одной таблице entries. Тип задаёт поле kind. Так не повторяются общие поля, но допустимые значения приходится ограничивать через CHECK. Учётная запись одна, заявки вынесены отдельно.

Таблица 2 - сущности базы

Таблица	**Ключ и назначение**
entries	id; уникальная пара kind и slug; материалы
admins	id = 1; соль и хеш пароля
inquiries	id; контакт, текст, согласие и статус заявки

Листинг 1 - фрагмент схемы entries

```
CREATE TABLE IF NOT EXISTS entries (
  id INTEGER PRIMARY KEY,
  kind TEXT NOT NULL
    CHECK (kind IN ('project','service','article')),
  slug TEXT NOT NULL,
  title TEXT NOT NULL,
  published INTEGER NOT NULL DEFAULT 0
    CHECK (published IN (0,1)),
  version INTEGER NOT NULL DEFAULT 1,
  UNIQUE(kind, slug)
);
```

PRIMARY KEY нумерует строку и не даёт двум записям занять один id. NOT NULL запрещает пустое обязательное поле. DEFAULT подставляет значение, если столбец не передали: новый материал рождается черновиком с version = 1. CHECK не пропускает посторонний kind и published вне 0 и 1. UNIQUE(kind, slug) хранит один адрес внутри типа: второй такой же проект получает ответ 409, а повтор услуги с тем же адресом проекту не мешает. Индекс entries_public построен по kind, published, sort_order и id: по нему сервер выбирает опубликованные карточки в заданном порядке.

К материалу добавлен учебный год. Базу я для этого не удалял. В store.mjs после создания таблиц читается PRAGMA table_info(entries). Если столбца study_year нет, выполняется ALTER TABLE entries ADD COLUMN study_year TEXT NOT NULL DEFAULT ''. Повторный запуск видит столбец и миграцию не повторяет. В вымышленных записях год равен 2026, на странице материала он выводится текстом.

3. Запуск

Сервер слушает только 127.0.0.1, порт 3090. Каталог данных - frontend-dev/data. При первом запуске создаются хеш пароля и вымышленные записи. При следующем запуске пароль заново не задаётся: в журнале нет строки о создании администратора, старые строки на месте.

Листинг 2 - запуск учебной копии

```
cd frontend-dev
node server/start.mjs
# http://127.0.0.1:3090/
# http://127.0.0.1:3090/admin/
```

Двойной щелчок по index.html открывает file://. В этом режиме сервер запрос не обрабатывает, и блок проектов остаётся подсказкой, как запустить Node.js. То же будет, если IIS отдаст папку как набор статических файлов: HTML приедет без подстановки из базы.

4. Одна запись в трёх состояниях

Для проверки взята вымышленная запись id = 5, slug chernovik-zametok, название «Черновик заметок к вёрстке». Рядом в базе четыре опубликованных проекта с sort_order 10, 20, 30 и 40, одна услуга и две заметки. У заметки «Заметка O'Neil» в названии есть апостроф: он сохранён параметром запроса и не сломал SQL. Публичный список её не показывает, потому что published = 0.

Пока запись 5 была черновиком, главная собирала четыре карточки. Прямой адрес /portfolio/chernovik-zametok/ отвечал 404 и текст материала не отдавал. В админке строка при этом видна.

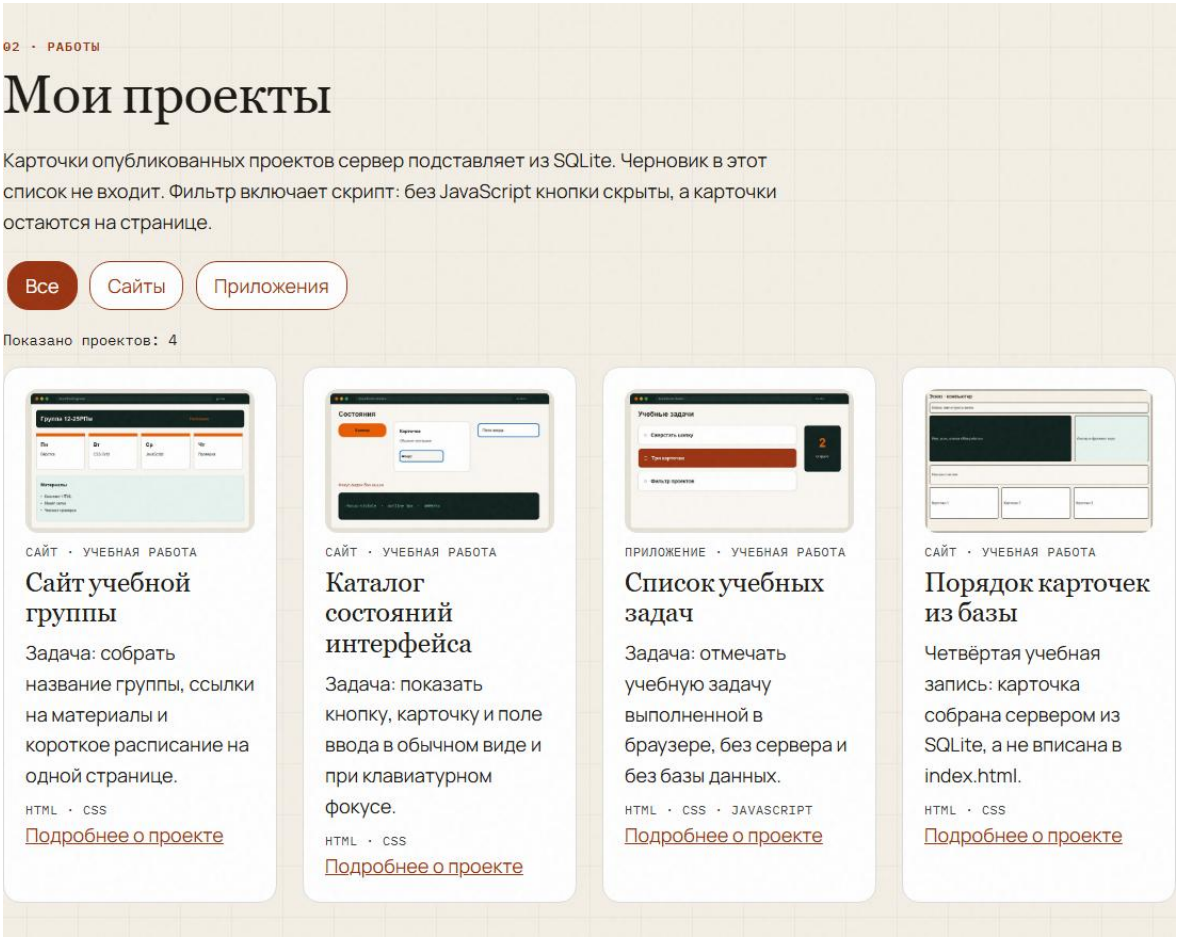


Рисунок 1 - Главная читает из базы четыре опубликованных проекта, черновика среди них нет

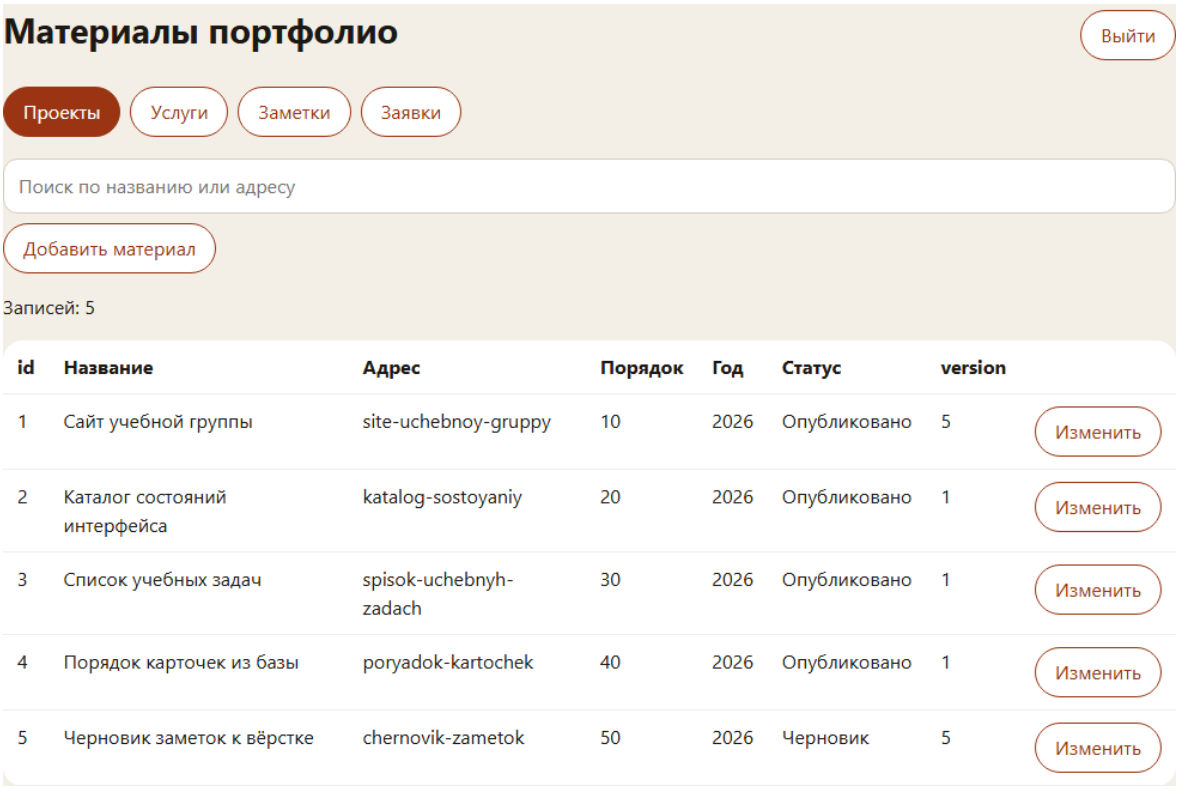


Рисунок 2 - В админке запись id = 5 остаётся черновиком, version = 5

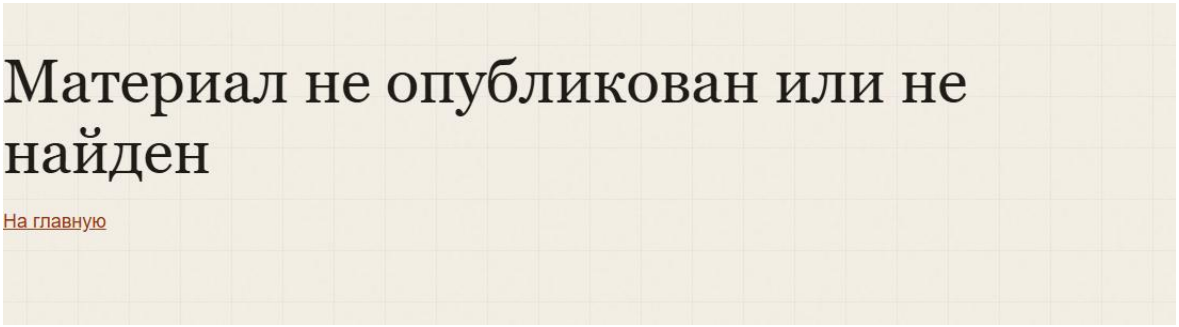


Рисунок 3 - Прямой адрес черновика содержимое не выдаёт

Затем ту же запись я опубликовал. Это UPDATE published = 1 и version = version + 1, не INSERT новой строки. На главной счётчик стал равен 5, карточка встала последней, потому что sort_order = 50. Страница /portfolio/chernovik-zametok/ открылась и показала учебный год 2026.

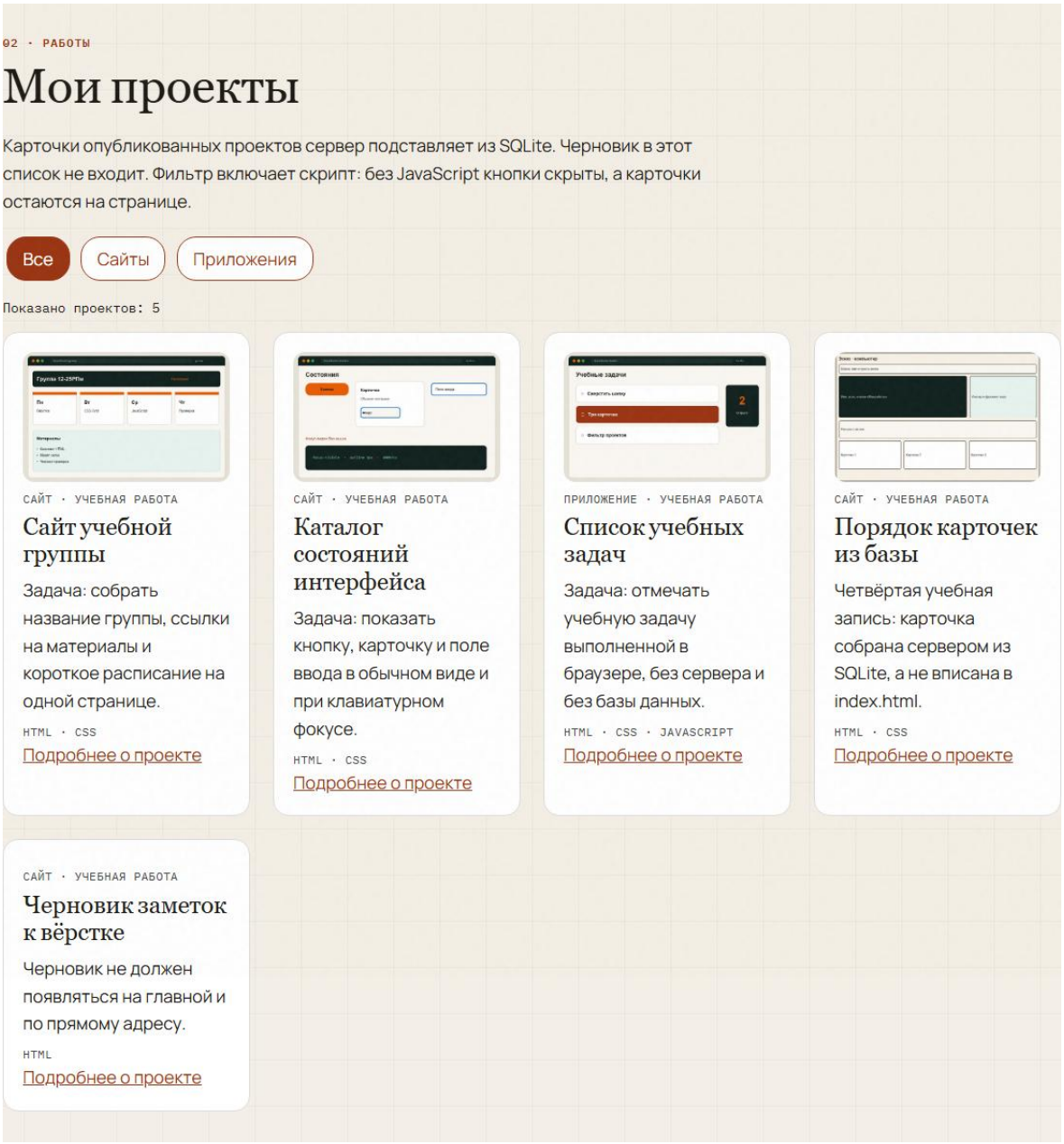


Рисунок 4 - Та же запись id = 5 появилась карточкой после публикации

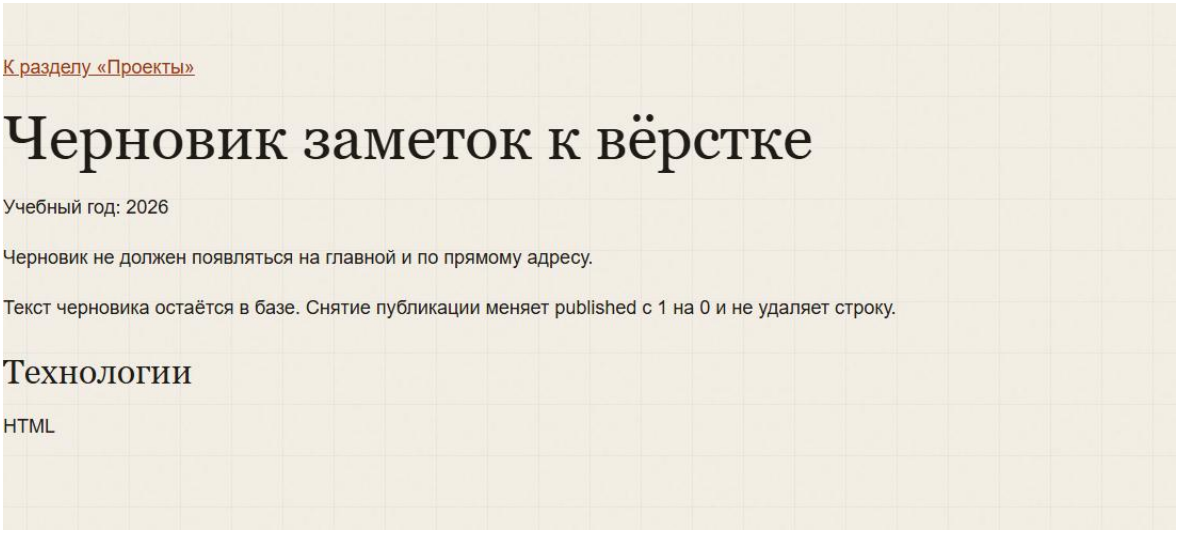
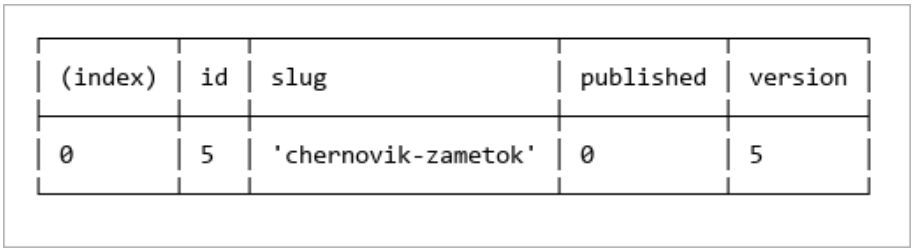


Рисунок 5 - Публичная страница материала, пока он был опубликован

Снятие публикации - тоже UPDATE: published возвращается в 0, version увеличивается, DELETE нет. Запись можно опубликовать снова. После этой проверки выборка по id = 5 показывает published = 0 и version = 5. Это текущее состояние базы, оно совпадает с рисунком 2. На главной снова четыре карточки, прямой адрес опять отвечает 404.

Листинг 3 - выборка той же записи

```
SELECT id, slug, published, version
FROM entries
WHERE id = 5;
```



(index)	id	slug	published	version
0	5	'chernovik-zametok'	0	5

Рисунок 6 - Вывод выборки: id = 5 на месте, публикация снята, version = 5

5. Чтение из Node.js

server/read-projects.mjs открывает базу только для чтения, готовит запрос с параметром kind и передаёт значение 'project' отдельно от текста SQL. Условие published = 1 отсекает черновик, поэтому в таблице четыре строки, хотя всего проектов пять. Если убрать это условие в своей копии сценария, в таблицу попадёт и черновик. Публичный маршрут сайта я для этого опыта не менял: он по-прежнему отдаёт только опубликованное.

Листинг 4 - server/read-projects.mjs

```
import { DatabaseSync } from 'node:sqlite';
import { join } from 'node:path';

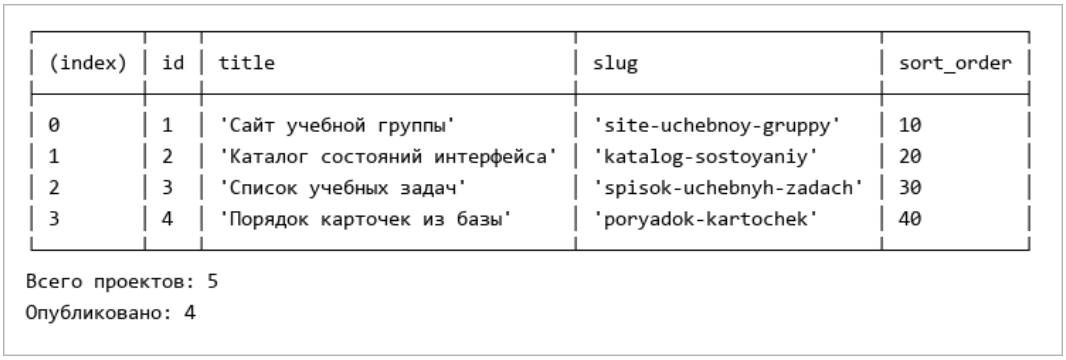
const folder = process.env.PORTFOLIO_DATA;
if (!folder) throw new Error('Задайте PORTFOLIO_DATA');

const db = new DatabaseSync(join(folder, 'portfolio.sqlite'), { readOnly: true });
db.exec('PRAGMA busy_timeout = 3000');

const query = db.prepare(`
  SELECT id, title, slug, sort_order
  FROM entries
  WHERE kind = ? AND published = 1
  ORDER BY sort_order, id
```

```
`);
console.table(query.all('project'));

const total = db.prepare(`
  SELECT COUNT(*) AS n FROM entries WHERE kind = 'project'
`).get();
const published = db.prepare(`
  SELECT COUNT(*) AS n FROM entries WHERE kind = 'project' AND published = 1
`).get();
console.log('Всего проектов:', total.n);
console.log('Опубликовано:', published.n);
db.close();
```



(index)	id	title	slug	sort_order
0	1	'Сайт учебной группы'	'site-uchebnoy-gruppy'	10
1	2	'Каталог состояний интерфейса'	'katalog-sostoyaniy'	20
2	3	'Список учебных задач'	'spisok-uchebnyh-zadach'	30
3	4	'Порядок карточек из базы'	'poryadok-kartochek'	40

Всего проектов: 5
Опубликовано: 4

Рисунок 7 - Четыре опубликованных проекта и два счётчика: всего 5, опубликовано 4

Числа сходятся с админкой: в списке проектов пять строк, статус «Опубликовано» у четырёх. Меньший sort_order идёт раньше. inspect.mjs в отдельном процессе показал те же id, slug, published и version, пароль и заявки он не печатает. Остановка сервера файл базы не удаляет.

6. Запись и откат транзакции

server/sql-experiment.mjs вставляет учебный черновик sql-experiment, меняет название и откатывает обе операции. Знаки вопроса - параметры. Название передаётся в run, а не склеивается со строкой SQL, поэтому апостроф остался бы текстом. Такой же приём стоит в store.mjs у INSERT и UPDATE.

Листинг 5 - server/sql-experiment.mjs

```
import { DatabaseSync } from 'node:sqlite';
import { join } from 'node:path';

const folder = process.env.PORTFOLIO_DATA;
if (!folder) throw new Error('Задайте PORTFOLIO_DATA');

const db = new DatabaseSync(join(folder, 'portfolio.sqlite'));
db.exec('PRAGMA busy_timeout = 3000');
```

```
db.exec('BEGIN');
try {
  const result = db.prepare(`
    INSERT INTO entries(kind, slug, title)
    VALUES (?, ?, ?)
  `).run('project', 'sql-experiment', 'SQL эксперимент');
  const id = Number(result.lastInsertRowid);
  db.prepare(`
    UPDATE entries SET title = ?, version = version + 1
    WHERE id = ?
  `).run('Название изменено', id);
  console.table(db.prepare(
    'SELECT id, title, published FROM entries WHERE id = ?'
  ).all(id));
} finally {
  db.exec('ROLLBACK');
  console.log('После ROLLBACK:');
  console.table(db.prepare(
    "SELECT id, title FROM entries WHERE slug = 'sql-experiment'"
  ).all());
  db.close();
}
```



Рисунок 8 - Внутри транзакции строка видна, после ROLLBACK выборка по slug пустая

Пока транзакция открыта, сценарий видит вставленную строку с новым названием и published = 0. ROLLBACK отменяет и INSERT, и UPDATE. COMMIT наоборот зафиксировал бы обе операции. После отката inspect и админка этой строки не показывают. Обновление версии в боевом сохранении тоже идёт одним UPDATE: WHERE id = ? AND version = ?, затем version = version + 1.

7. От формы админки до SQL

Обработчик entry-form в admin.js читает поля и вызывает API. app.mjs проверяет сеанс, заголовок x-csrf-token, название, длину строк и ссылку. javascript: и data: отклоняются до SQL. store.mjs затем выполняет запрос. Пустое название получает ответ 400, повтор пары kind и slug - 409.

Таблица 3 - основные маршруты

Запрос	**Назначение**
POST /api/login	Проверить пароль и открыть сеанс
GET /api/admin/entries	Список материалов после входа
POST /api/admin/entries	Создать материал
PUT /api/admin/entries/ID	Изменить материал с проверкой version
GET /api/projects	Опубликованные проекты в JSON
POST /api/inquiries	Сохранить заявку, если согласие истинно
PUT /api/admin/inquiries/ID	Сменить статус заявки

Поле version нужно, чтобы вторая вкладка не затёрла чужую правку. Запрос обновляет строку только если id и version совпали с тем, что было на экране. Если править лишь по id, в базу ушло бы устаревшее название и закрыло бы уже сохранённое описание. В проверке одна вкладка держала старую version, другая успела сохранить описание. Вторая вкладка получила 409, форма показала конфликт, прежнее название «Сайт учебной группы» осталось. Cookie сам права не даёт: без сеанса GET /api/admin/entries отвечает 401, а чужой CSRF-токен сервер не принимает.

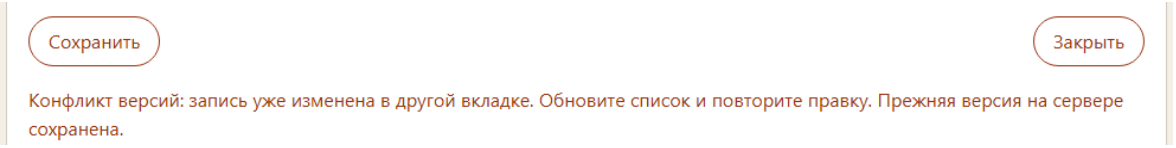


Рисунок 9 - Ответ 409 при сохранении устаревшей version

Текст со строкой <script> я сохранял как черновик заметки и на время открывал публичную страницу. В HTML он попал как экранированный текст, alert не выполнялся. После проверки публикацию снял. Физического удаления в админке нет.

8. Заявки

На главной форма из server/site.js шлёт POST /api/inquiries. Запись в базе не означает отправку письма: почты и Telegram в этой версии нет, рядом по-прежнему стоит mailto. Сервер сам проверяет consent. При значении false заявка не пишется, ответ 400. Для вымышленной заявки согласие было истинным, статус «Новая». Кнопки админки меняют его на «В работе» и «Завершена», после обновления страницы значение остаётся. Одинаковые запросы с одного адреса режутся ограничением частоты.

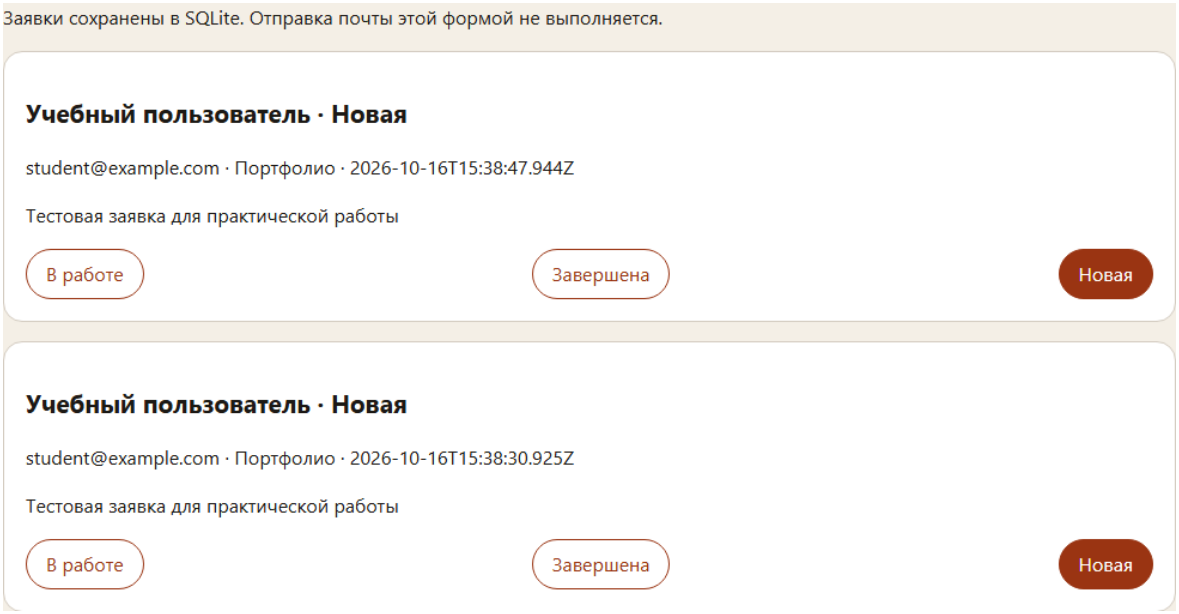


Рисунок 10 - Учебные заявки в админке, почта не отправляется

Услуга и заметка читаются теми же запросами, но с другим kind. Они выведены на /services/ и /blog/, на главную сетку проектов не попадают.

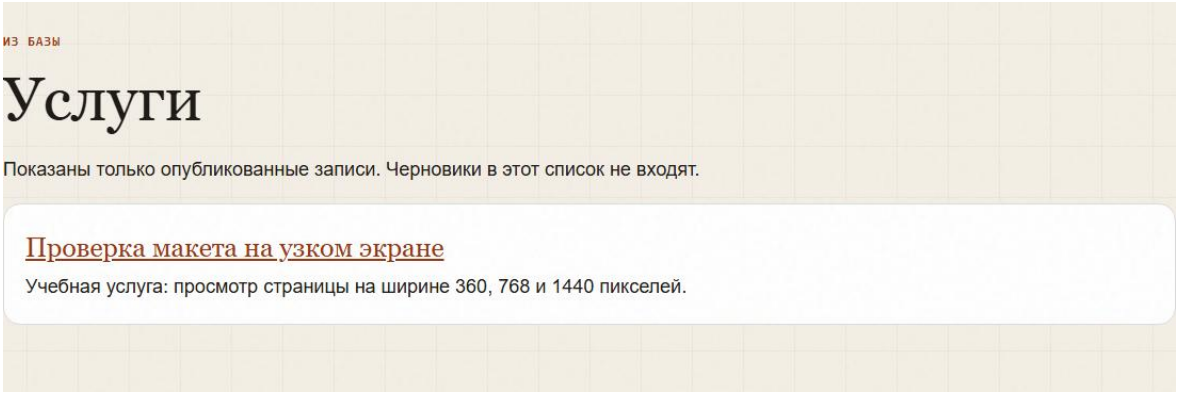


Рисунок 11 - Опубликованная услуга из той же таблицы entries

9. Матрица проверок

Проверки шли на вымышленных записях. После матрицы сервер был остановлен и запущен снова с тем же файлом базы.

Таблица 4 - фактические результаты

Действие	Что получилось
Сохранить и перезапустить сервер	Строки на месте. Старый сеанс отвечает 401, нужен новый вход
Создать или оставить черновик	В админке виден, на главной и по прямому адресу его нет
Повторить kind и slug	Ответ 409, второй строки нет
Пустое название	Ответ 400, в базу не пишется
Ссылка javascript:	Сервер отклоняет ссылку до SQL
Текст с <script>	На странице это текст, скрипт не выполняется
API админки без входа	Ответ 401
Устаревшая version	Ответ 409, уже сохранённая правка не затёрта
Снять публикацию	Страница материала больше не отдаёт текст, строка в базе есть

Файл базы лежит в data, не в images и не в css. Пока сервер работает, SQLite может держать рядом файлы WAL, поэтому копия одного .sqlite в этот момент ненадёжна. Для учебной копии сервер останавливается, и копируется весь каталог data. Пароли и копия базы в отчёт не кладутся.

10. IIS и самостоятельные задания

IIS сам по себе код Node.js не исполняет. Для публичной площадки процесс Node.js должен работать отдельно, а IIS с URL Rewrite и ARR передавать ему запросы всего сайта. Если проксировать только API, HTML останется статическим и карточки из базы не соберутся. В этой работе хватает локального адреса 127.0.0.1. Службу Windows, HTTPS и публичный доступ к учебной базе я не включал.

Самостоятельное задание 1. Четвёртый проект «Порядок карточек из базы» описан своими словами, картинка взята из images/sketch-desktop.png, sort_order = 40. Запись id = 5 я опубликовал и снова скрыл. Одна и та же строка видна в админке, в выборке WHERE id = 5 и, пока она опубликована, на сайте.

Самостоятельное задание 2. В read-projects.mjs два запроса SELECT COUNT(*): все проекты и только опубликованные. Получилось 5 и 4, как в админке на рисунке 2 и в выводе на рисунке 7.

Самостоятельное задание 3. Строка tech неудобна: одно и то же название можно записать по-разному, по одной технологии нельзя отобрать карточки и нельзя посчитать, сколько работ использует CSS. Для связи многие-ко-многим подошли бы таблицы technologies (id, name UNIQUE) и entry_technologies (entry_id, technology_id) с составным первичным ключом и внешними ключами на entries и technologies. В готовой схеме эту связь я не строил: задание просит объяснить недостаток, а не менять рабочую таблицу.

Дополнительно к материалу добавлен учебный год. Схема чтения дополнена столбцом study_year, форма админки его редактирует, сервер принимает только значение вроде 2026, публичная страница печатает его как текст. Существующий файл базы не удалялся: столбец добавляет повторяемый ALTER TABLE.

11. Вывод

Браузер не открывает SQLite напрямую, потому что файл базы нельзя отдать посетителю и нельзя проверить права в самом браузере. Черновик отличается от удалённой записи тем, что строка остаётся, а публичный запрос её не выбирает. Параметры SQL отделяют данные от текста запроса. Поле version не даёт устаревшей вкладке затереть новую правку. Cookie только приносит номер сеанса, а решение о доступе принимает сервер. Адрес file:// не показывает данные базы, потому что запрос не доходит до Node.js. Между IIS и Node.js нужен обратный прокси на весь сайт.

Учебное портфолио в папке frontend-dev открывается по http://127.0.0.1:3090/ и берёт карточки из локальной базы с вымышленными записями. После перезапуска сервера записи сохранились.